

Microsoft Fabric

WMS SQL-First Medallion Architecture

Bronze · Silver · Gold · Streaming Pipeline

19 Source Tables · 47 Delta Tables · ~18 Lines of PySpark · 64 SQL Cells

Part 1 — Bronze nb_bronze_ingest_sql

19 tables · 21 cells · SQL read_files() CTAS

Part 2 — Silver nb_silver_transform_sql

15 tables · 16 cells · CTE dedup + typed casts

Part 3 — Gold nb_gold_build_sql

12 tables · 17 cells · star schema + OPTIMIZE

Part 4 — Streaming nb_iot_stream_sql

1 table · 10 cells · Delta readStream + SQL window

All transformation logic in SQL · Streaming via Delta readStream/writeStream

Contents

■ Part 1 — Bronze Ingestion

- Cell 1: Schema Bootstrap (Python — 3 lines only)
- Cells 2–20: read_files() CTAS for all 19 tables
- Cell 21: Full Bronze Validation (UNION ALL audit)

■ Part 2 — Silver Transform

- Cells 1–15: CTE dedup + ROW_NUMBER() + type casts
- Includes: inventory_transactions, locations (added)
- Cell 16: Full Silver Validation (null + dupe check)

■ Part 3 — Gold Batch

- Cells 1–4: Dimension tables (date, customer, product, warehouse)
- Cells 5–9: Fact tables (orders, inventory, shipments, labor, dock)
- Cells 10–12: KPI Marts (supplier, returns, IoT daily)
- Cells 13–15: OPTIMIZE + ZORDER + VACUUM (pure SQL)
- Cells 16–17: Validation + revenue sanity check

■ Part 4 — IoT Streaming + Pipeline

- Cell 1: CREATE TABLE gold_kpi_iot_stream (SQL schema)
- Cell 2: readStream + temp view registration (Python)
- Cell 3: SQL tumbling window aggregation (1-hour buckets)
- Cell 4: writeStream → gold.gold_kpi_iot_stream (Python)
- Cells 5–10: Monitor, stop, spot-check, alert queries
- Data Factory pipeline wiring (batch + streaming jobs)
- Power BI streaming tile setup + DAX measures

■ PART 1

Bronze Ingestion

Raw CSV → Delta via read_files() CTAS

`nb_bronze_ingest_sql`

- 19 Delta tables written to bronze schema
- read_files() TVF — no temp views, no PySpark loops
- delta.enableChangeDataFeed = true on raw_iot_events
- UNION ALL audit validates all 19 PKs in one query
- PySpark: 3 lines (schema bootstrap only)

WMS SQL-First Medallion — Part 1 of 4: BRONZE

Microsoft Fabric · Notebook: nb_bronze_ingest_sql

Philosophy:

- All 19 tables ingested via SQL read_files() TVF — zero PySpark loops
- Python is used ONCE: schema bootstrap only
- Each table is its own SQL cell for granular error isolation
- delta.enableChangeDataFeed = true on raw_iot_events so the streaming notebook (Part 4) can read it incrementally via readStream
- Run Cell 1, then Cells 2-20 in any order, then Cell 21 for validation

```

-- Create schema if not exists
CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;

```

```

-- Create schema if not exists
CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;

```

```

-- Create schema if not exists
CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;

```

CELL 1 – Schema Bootstrap (Python – only Python in this notebook)

```

for schema in ['bronze', 'silver', 'gold']:
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema}")
print("Schemas ready: bronze | silver | gold")

```

```

-- Create schema if not exists
CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;

```

```

-- Create schema if not exists
CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;

```

CELL 2 – raw_orders (50,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_orders
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,

```

```
current_timestamp()) AS _loaded_at
FROM read_files(
  'Files/raw/raw_orders.csv',
  format      => 'csv',
  header      => 'true',
  inferSchema => 'true',
  nullValue   => ''
)
WHERE order_id IS NOT NULL;

SELECT 'bronze.raw_orders' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_orders;
```

Group	Percentage
Democrat	93%
Republican	83%

Group	Percentage
Democrat	85%
Republican	60%

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100

CELL 3 – raw_order_lines (128,899 rows)

```
CREATE OR REPLACE TABLE bronze.raw_order_lines
USING DELTA

TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_order_lines.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE line_id IS NOT NULL;

SELECT 'bronze.raw_order_lines' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_order_lines;
```

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	5
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	---

Government	Percentage
Current government	85%
Previous government	15%

CELL 4 – raw_customers (500 rows)

```
CREATE OR REPLACE TABLE bronze.raw_customers
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
```

```
FROM read_files(
  'Files/raw/raw_customers.csv',
  format      => 'csv',
  header      => 'true',
  inferSchema => 'true',
  nullValue   => ''
)
WHERE customer_id IS NOT NULL;

SELECT 'bronze.raw_customers' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_customers;
```

[illegible]

Response	Percentage
Yes	85%
No	15%

CELL 5 – raw_products (1,000 rows)

```
CREATE OR REPLACE TABLE bronze.raw_products
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_products.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE sku IS NOT NULL;

SELECT 'bronze.raw_products' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_products;
```

Response	Percentage
Yes	85%
No	15%

Response	Percentage
Yes	85%
No	15%

CELL 6 – raw_inventory (10,000 rows)

```
CREATE OR REPLACE TABLE bronze.raw_inventory
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
```

```

    'Files/raw/raw_inventory.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE inventory_id IS NOT NULL;

SELECT 'bronze.raw_inventory' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_inventory;

```



CELL 7 – raw_inventory_transactions (50,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_inventory_transactions
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_inventory_transactions.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE txn_id IS NOT NULL;

SELECT 'bronze.raw_inventory_transactions' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_inventory_transactions;

```



CELL 8 – raw_shipments (30,053 rows)

```

CREATE OR REPLACE TABLE bronze.raw_shipments
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_shipments.csv',

```

```
format      => 'csv',
header      => 'true',
inferSchema => 'true',
nullValue   => ''
)
WHERE shipment_id IS NOT NULL;

SELECT 'bronze.raw_shipments' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_shipments;
```

Two horizontal bars are shown. The top bar is blue and represents 100% of the population. The bottom bar is red and represents 10% of the population.

CELL 9 – raw_carriers (10 rows)

```
CREATE OR REPLACE TABLE bronze.raw_carriers
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_carriers.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE carrier_id IS NOT NULL;

SELECT 'bronze.raw_carriers' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_carriers;
```

Gender	Percentage
Men	80%
Women	60%

Response	Percentage
Yes	90%
No	10%

Response	Percentage
Yes	75%
No	25%

CELL 10 – raw_warehouses (10 rows)

```
CREATE OR REPLACE TABLE bronze.raw_warehouses
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_warehouses.csv',
    format      => 'csv',
```



```

        header      => 'true',
        inferSchema => 'true',
        nullValue   => ''
    )
WHERE warehouse_id IS NOT NULL;

```

```

SELECT 'bronze.raw_warehouses' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_warehouses;

```



CELL 11 – raw_locations (2,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_locations
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_locations.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE location_id IS NOT NULL;

```

```

SELECT 'bronze.raw_locations' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_locations;

```



CELL 12 – raw_purchase_orders (8,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_purchase_orders
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_purchase_orders.csv',
    format      => 'csv',
    header      => 'true',

```

```

        inferSchema => 'true',
        nullValue   => ''
    )
WHERE po_id IS NOT NULL;

SELECT 'bronze.raw_purchase_orders' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_purchase_orders;

```

[illegible][illegible]

CELL 13 – raw_suppliers (80 rows)

```
CREATE OR REPLACE TABLE bronze.raw_suppliers
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_suppliers.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE supplier_id IS NOT NULL;

SELECT 'bronze.raw_suppliers' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_suppliers;
```

[illegible]

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	5
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	---

CELL 14 – raw_supplier_performance (2,400 rows)

```
CREATE OR REPLACE TABLE bronze.raw_supplier_performance
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_supplier_performance.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
```

```

        nullValue => ''
    )
WHERE perf_id IS NOT NULL;

SELECT 'bronze.raw_supplier_performance' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_supplier_performance;

```

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	5
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	---

[illegible]

CELL 15 – raw_returns (5,000 rows)

```
CREATE OR REPLACE TABLE bronze.raw_returns
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_returns.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE return_id IS NOT NULL;

SELECT 'bronze.raw_returns' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_returns;
```

Response	Percentage
Yes	90%
No	10%

Response	Percentage
Yes	75%
No	25%

Response	Percentage
Yes	85%
No	15%

CELL 16 – raw_employees (200 rows)

```
CREATE OR REPLACE TABLE bronze.raw_employees
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_employees.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
```

```
)
WHERE employee_id IS NOT NULL;

SELECT 'bronze.raw_employees' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_employees;
```

CELL 17 – raw_labor_shifts (25,000 rows)

```
CREATE OR REPLACE TABLE bronze.raw_labor_shifts
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_labor_shifts.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE shift_id IS NOT NULL;

SELECT 'bronze.raw_labor_shifts' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_labor_shifts;
```

CELL 18 – raw_dock_activity (15,000 rows)

```
CREATE OR REPLACE TABLE bronze.raw_dock_activity
USING DELTA

TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_dock_activity.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
```

```
WHERE dock_id IS NOT NULL;
```

```
SELECT 'bronze.raw_dock_activity' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_dock_activity;
```



IMPORTANT: Two extra TBLPROPERTIES here:

delta.enableChangeDataFeed = true

→ **allows the streaming notebook (Part 4) to read this table**

incrementally using spark.readStream.format("delta")

delta.autoOptimize.autoCompact = true

→ **keeps the file layout clean as new event batches append**



CELL 19 – raw_iot_events (100,000 rows)

```
CREATE OR REPLACE TABLE bronze.raw_iot_events
USING DELTA
TBLPROPERTIES (
  'delta.autoOptimize.optimizeWrite' = 'true',
  'delta.autoOptimize.autoCompact'   = 'true',
  'delta.enableChangeDataFeed'       = 'true'
)
AS
SELECT
  *,
  current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_iot_events.csv',
  format      => 'csv',
  header      => 'true',
  inferSchema => 'true',
  nullValue   => ''
)
WHERE event_id IS NOT NULL;

SELECT 'bronze.raw_iot_events' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_iot_events;
```



CELL 20 – raw_date_dim (1,096 rows)

```

CREATE OR REPLACE TABLE bronze.raw_date_dim
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM read_files(
    'Files/raw/raw_date_dim.csv',
    format      => 'csv',
    header      => 'true',
    inferSchema => 'true',
    nullValue   => ''
)
WHERE date_key IS NOT NULL;

SELECT 'bronze.raw_date_dim' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_date_dim;

```

Row counts + PK null audit across all 19 tables in one query.

All rows in the status column must read 'OK' before proceeding

to Silver (Part 2).

CELL 21 – Full Bronze Validation

```

WITH audit AS (
    SELECT 'raw_orders'           AS tbl, 'order_id'       AS pk,
           COUNT(*)              AS rows,
           SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
    FROM bronze.raw_orders

    UNION ALL SELECT 'raw_order_lines', 'line_id',
                     COUNT(*), SUM(CASE WHEN line_id IS NULL THEN 1 ELSE 0 END)
    FROM bronze.raw_order_lines

    UNION ALL SELECT 'raw_customers', 'customer_id',
                     COUNT(*), SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END)
    FROM bronze.raw_customers

    UNION ALL SELECT 'raw_products', 'sku',
                     COUNT(*), SUM(CASE WHEN sku IS NULL THEN 1 ELSE 0 END)
    FROM bronze.raw_products

```

```

UNION ALL SELECT 'raw_inventory', 'inventory_id',
    COUNT(*), SUM(CASE WHEN inventory_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_inventory

UNION ALL SELECT 'raw_inventory_transactions', 'txn_id',
    COUNT(*), SUM(CASE WHEN txn_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_inventory_transactions

UNION ALL SELECT 'raw_shipments', 'shipment_id',
    COUNT(*), SUM(CASE WHEN shipment_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_shipments

UNION ALL SELECT 'raw_carriers', 'carrier_id',
    COUNT(*), SUM(CASE WHEN carrier_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_carriers

UNION ALL SELECT 'raw_warehouses', 'warehouse_id',
    COUNT(*), SUM(CASE WHEN warehouse_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_warehouses

UNION ALL SELECT 'raw_locations', 'location_id',
    COUNT(*), SUM(CASE WHEN location_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_locations

UNION ALL SELECT 'raw_purchase_orders', 'po_id',
    COUNT(*), SUM(CASE WHEN po_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_purchase_orders

UNION ALL SELECT 'raw_suppliers', 'supplier_id',
    COUNT(*), SUM(CASE WHEN supplier_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_suppliers

UNION ALL SELECT 'raw_supplier_performance', 'perf_id',
    COUNT(*), SUM(CASE WHEN perf_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_supplier_performance

UNION ALL SELECT 'raw_returns', 'return_id',
    COUNT(*), SUM(CASE WHEN return_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_returns

UNION ALL SELECT 'raw_employees', 'employee_id',
    COUNT(*), SUM(CASE WHEN employee_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_employees

UNION ALL SELECT 'raw_labor_shifts', 'shift_id',
    COUNT(*), SUM(CASE WHEN shift_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_labor_shifts

UNION ALL SELECT 'raw_dock_activity', 'dock_id',
    COUNT(*), SUM(CASE WHEN dock_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_dock_activity

UNION ALL SELECT 'raw_iot_events', 'event_id',
    COUNT(*), SUM(CASE WHEN event_id IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_iot_events

UNION ALL SELECT 'raw_date_dim', 'date_key',
    COUNT(*), SUM(CASE WHEN date_key IS NULL THEN 1 ELSE 0 END)
FROM bronze.raw_date_dim
)
SELECT
    tbl,

```

CELL 22 – Optional: Describe a Table to Verify Schema Inference

```
-- Swap table name as needed to spot-check inferred types
DESCRIBE TABLE bronze.raw_orders;
```

Response	Percentage
U.S. should take more action	90%
U.S. should not take more action	10%

Response	Percentage
U.S. should take more action	75%
U.S. should not take more action	25%

NOTES FOR FABRIC DEPLOYMENT

90%

75%

1. read_files() path:

'Files/raw/...' is the relative path within your Lakehouse Files

section. Upload all 19 CSVs to that folder before running.

If you use ADLS Gen2 directly, replace with the full abfss:// URI:

abfss://@.dfs.core.windows.net/rawl.csv

2. inferSchema vs. explicit schema:

inferSchema works for initial builds. For production, replace with an explicit SCHEMA clause to prevent silent type drift between runs:


```
FROM read_files('...', format => 'csv', header => 'true',  
schema => 'order_id STRING, order_date STRING, ...')
```

3. Overwrite safety:

CREATE OR REPLACE TABLE is a full overwrite each pipeline run.

For incremental Bronze appends (e.g., daily new orders), replace

with **INSERT INTO ... SELECT ... WHERE order_date > (SELECT MAX(...))**

4. raw_iot_events — Change Data Feed:

delta.enableChangeDataFeed = true is required so Part 4's

streaming notebook can read this table with **spark.readStream**.

Do NOT disable this property after the stream is running.

5. Pipeline order: Bronze must complete fully before Silver starts.

Wire notebooks in Data Factory as:

[nb_bronze_ingest_sql] → success → [nb_silver_transform_sql]



■ PART 2

Silver Transform

Curated, typed, deduplicated tables

`nb_silver_transform_sql`

- 15 Delta tables written to silver schema
- ROW_NUMBER() window deduplication — no .dropDuplicates()
- All type casts, derived columns, and joins in pure SQL CTEs
- Reference lookups from Bronze to avoid inter-Silver dependencies
- PySpark: 0 lines

Microsoft Fabric · Notebook: nb_silver_transform_sql

- Zero PySpark. Every transform is a SQL CTAS with CTE chains.
- Deduplication uses ROW_NUMBER() window function — no .dropDuplicates()
- Reference lookups (carriers, suppliers, products, warehouses) are joined inline inside the CTE, reading directly from Bronze to avoid inter-Silver ordering dependencies.
- All derived columns (margins, rates, flags, buckets) are CASE WHEN or arithmetic expressions inside the SELECT — no Python lambdas.
- Two tables missing from the original Silver spec are included here: silver_inventory_transactions and silver_locations.
- Run Bronze validation (Part 1, Cell 21) before this notebook.

Response	Percentage
U.S. should take more action	88%
U.S. should not take more action	12%

Response	Percentage
U.S. should take more action	75%
U.S. should not take more action	25%

90%

75%

Source: `bronze.raw_orders`

Key derived cols: days_to_required, order_month, order_year

Gender	Percentage
Men	85%
Women	90%

CELL 1 – silver_orders

```
CREATE OR REPLACE TABLE silver.silver_orders
USING DELTA
```

```

TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY order_id
            ORDER BY _loaded_at DESC
        ) AS rn
    FROM bronze.raw_orders
    WHERE order_id IS NOT NULL
)
SELECT
    order_id,
    customer_id,
    warehouse_id,
    TO_DATE(order_date)                AS order_date,
    TO_DATE(required_date)             AS required_date,
    CAST(order_value AS DOUBLE)        AS order_value,
    CAST(total_lines AS INT)           AS total_lines,
    UPPER(TRIM(status))                AS status,
    UPPER(TRIM(priority))              AS priority,
    channel,
    DATEDIFF(
        TO_DATE(required_date),
        TO_DATE(order_date)
    )                                  AS days_to_required,
    DATE_FORMAT(TO_DATE(order_date), 'yyyy-MM') AS order_month,
    YEAR(TO_DATE(order_date))             AS order_year,
    current_timestamp()                 AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'silver.silver_orders' AS tbl, COUNT(*) AS rows FROM silver.silver_orders;

```



Source: bronze.raw_order_lines + bronze.raw_products (lookup)

Key derived cols: fulfillment_rate, line_margin, category/subcategory



CELL 2 – silver_order_lines

```

CREATE OR REPLACE TABLE silver.silver_order_lines
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY line_id
            ORDER BY _loaded_at DESC
        ) AS rn
    FROM bronze.raw_order_lines

```

```

        WHERE line_id IS NOT NULL
    ),
    typed AS (
        SELECT
            line_id,
            order_id,
            sku,
            CAST(qty_ordered AS INT) AS qty_ordered,
            CAST(qty_fulfilled AS INT) AS qty_fulfilled,
            CAST(qty_backordered AS INT) AS qty_backordered,
            CAST(unit_price AS DOUBLE) AS unit_price,
            CAST(line_value AS DOUBLE) AS line_value,
            TO_DATE(fulfilled_date) AS fulfilled_date,
            CAST(is_backordered AS BOOLEAN) AS is_backordered,
            CASE
                WHEN CAST(qty_ordered AS INT) > 0
                THEN CAST(qty_fulfilled AS DOUBLE) / CAST(qty_ordered AS DOUBLE)
                ELSE 0.0
            END AS fulfillment_rate
        FROM deduped
        WHERE rn = 1
    ),
    with_product AS (
        SELECT
            t.*,
            p.unit_cost,
            p.category,
            p.subcategory,
            -- line margin = revenue minus cost of goods fulfilled
            t.line_value - (t.qty_fulfilled * p.unit_cost) AS line_margin
        FROM typed t
        LEFT JOIN bronze.raw_products p
            ON t.sku = p.sku
    )
    SELECT
        *,
        current_timestamp() AS _loaded_at
    FROM with_product;

SELECT 'silver.silver_order_lines' AS tbl, COUNT(*) AS rows FROM silver.silver_order_lines;

```



Source: bronze.raw_customers

Key derived cols: segment/account_status normalized, customer_since_year



CELL 3 – silver_customers

```

CREATE OR REPLACE TABLE silver.silver_customers
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS

```

```

WITH deduped AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY _loaded_at DESC
           ) AS rn
    FROM bronze.raw_customers
    WHERE customer_id IS NOT NULL
)
SELECT
    customer_id,
    first_name,
    last_name,
    email,
    phone,
    UPPER(TRIM(segment))      AS segment,
    UPPER(TRIM(account_status)) AS account_status,
    CAST(credit_limit AS DOUBLE) AS credit_limit,
    city,
    state,
    region,
    country,
    TO_DATE(created_date)      AS created_date,
    YEAR(TO_DATE(created_date)) AS customer_since_year,
    current_timestamp()        AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'silver.silver_customers' AS tbl, COUNT(*) AS rows FROM silver.silver_customers;

```

Source: bronze.raw_products

Key derived cols: margin_pct, volume_cuft

CELL 4 – silver_products

```

CREATE OR REPLACE TABLE silver.silver_products
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY sku
               ORDER BY _loaded_at DESC
           ) AS rn
    FROM bronze.raw_products
    WHERE sku IS NOT NULL
)
SELECT
    sku,

```

```

product_name,
category,
subcategory,
supplier_id,
CAST(unit_cost AS DOUBLE) AS unit_cost,
CAST(unit_price AS DOUBLE) AS unit_price,
CAST(weight_kg AS DOUBLE) AS weight_kg,
CAST(length_cm AS DOUBLE) AS length_cm,
CAST(width_cm AS DOUBLE) AS width_cm,
CAST(height_cm AS DOUBLE) AS height_cm,
CAST(reorder_point AS INT) AS reorder_point,
CAST(reorder_qty AS INT) AS reorder_qty,
CAST(lead_time_days AS INT) AS lead_time_days,
CAST(is_hazmat AS BOOLEAN) AS is_hazmat,
CAST(is_perishable AS BOOLEAN) AS is_perishable,
-- margin as a fraction of price
CASE
  WHEN CAST(unit_price AS DOUBLE) > 0
  THEN (CAST(unit_price AS DOUBLE) - CAST(unit_cost AS DOUBLE))
    / CAST(unit_price AS DOUBLE)
  ELSE 0.0
END
AS margin_pct,
-- volume in cubic feet (1 cubic foot = 28316.8 cubic cm)
(CAST(length_cm AS DOUBLE)
 * CAST(width_cm AS DOUBLE)
 * CAST(height_cm AS DOUBLE)
 / 28316.8
AS volume_cuft,
current_timestamp()
AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'silver.silver_products' AS tbl, COUNT(*) AS rows FROM silver.silver_products;

```



Source: bronze.raw_inventory + bronze.raw_products (lookup)

Key derived cols: inventory_value, below_reorder flag, utilization_rate



CELL 5 – silver_inventory

```

CREATE OR REPLACE TABLE silver.silver_inventory
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY inventory_id
      ORDER BY _loaded_at DESC
    ) AS rn
  FROM bronze.raw_inventory
  WHERE inventory_id IS NOT NULL

```

```

        AND sku IS NOT NULL
    ),
    typed AS (
        SELECT
            inventory_id,
            sku,
            warehouse_id,
            location_id,
            CAST(qty_on_hand AS INT) AS qty_on_hand,
            CAST(qty_reserved AS INT) AS qty_reserved,
            CAST(qty_available AS INT) AS qty_available,
            CAST(qty_in_transit AS INT) AS qty_in_transit,
            CAST(unit_cost AS DOUBLE) AS unit_cost,
            CAST(reorder_point AS INT) AS reorder_point,
            TO_TIMESTAMP(last_counted_at) AS last_counted_at,
            TO_TIMESTAMP(last_received_at) AS last_received_at
        FROM deduped
        WHERE rn = 1
    ),
    enriched AS (
        SELECT
            t.*,
            -- financial exposure of on-hand stock
            t.qty_on_hand * t.unit_cost AS inventory_value,
            -- true when available qty has dropped below reorder threshold
            t.qty_available < t.reorder_point AS below_reorder,
            -- ratio of on-hand to reorder point; null when reorder_point = 0
            CASE
                WHEN t.reorder_point > 0
                THEN CAST(t.qty_on_hand AS DOUBLE) / t.reorder_point
                ELSE NULL
            END AS utilization_rate,
            p.product_name,
            p.category,
            p.subcategory
        FROM typed t
        LEFT JOIN bronze.raw_products p
            ON t.sku = p.sku
    )
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM enriched;

SELECT 'silver.silver_inventory' AS tbl, COUNT(*) AS rows FROM silver.silver_inventory;

```



Source: bronze.raw_inventory_transactions

Key derived cols: txn_date, txn_month for time-series joins

(Not in original spec — included here for completeness)

CELL 6 – silver_inventory_transactions

```

CREATE OR REPLACE TABLE silver.silver_inventory_transactions
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY txn_id
               ORDER BY _loaded_at DESC
           ) AS rn
    FROM bronze.raw_inventory_transactions
    WHERE txn_id IS NOT NULL
)
SELECT
    txn_id,
    inventory_id,
    sku,
    warehouse_id,
    location_id,
    txn_type,
    reference_id,
    reference_type,
    CAST(qty_change AS INT)           AS qty_change,
    CAST(qty_before AS INT)          AS qty_before,
    CAST(qty_after AS INT)           AS qty_after,
    -- sign check: flag transactions where qty math doesn't reconcile
    CASE
        WHEN CAST(qty_before AS INT) + CAST(qty_change AS INT)
            <> CAST(qty_after AS INT)
        THEN TRUE ELSE FALSE
    END                               AS qty_mismatch_flag,
    TO_TIMESTAMP(txn_timestamp)       AS txn_timestamp,
    TO_DATE(txn_timestamp)            AS txn_date,
    DATE_FORMAT(
        TO_DATE(txn_timestamp), 'yyyy-MM'
    )                                 AS txn_month,
    performed_by,
    current_timestamp()               AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'silver.silver_inventory_transactions' AS tbl,
       COUNT(*) AS rows
FROM silver.silver_inventory_transactions;

```

Source: bronze.raw_shipments + bronze.raw_carriers (lookup)

Key derived cols: delay_days, cost_per_lb, carrier attributes

CELL 7 – silver_shipments

```

CREATE OR REPLACE TABLE silver.silver_shipments
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY shipment_id
            ORDER BY _loaded_at DESC
        ) AS rn
    FROM bronze.raw_shipments
    WHERE shipment_id IS NOT NULL
),
typed AS (
    SELECT
        shipment_id,
        order_id,
        carrier_id,
        origin_warehouse_id,
        TO_DATE(ship_date) AS ship_date,
        TO_DATE(estimated_delivery) AS estimated_delivery,
        TO_DATE(actual_delivery) AS actual_delivery,
        UPPER(TRIM(status)) AS status,
        CAST(shipping_cost AS DOUBLE) AS shipping_cost,
        CAST(weight_lbs AS DOUBLE) AS weight_lbs,
        CAST(num_packages AS INT) AS num_packages,
        CAST(is_on_time AS BOOLEAN) AS is_on_time,
        -- positive = late, negative = early, 0 = exact
        CASE
            WHEN actual_delivery IS NOT NULL
            AND estimated_delivery IS NOT NULL
            THEN DATEDIFF(
                TO_DATE(actual_delivery),
                TO_DATE(estimated_delivery))
            ELSE 0
        END AS delay_days,
        -- shipping cost efficiency per pound
        CASE
            WHEN CAST(weight_lbs AS DOUBLE) > 0
            THEN CAST(shipping_cost AS DOUBLE) / CAST(weight_lbs AS DOUBLE)
            ELSE NULL
        END AS cost_per_lb
    FROM deduped
    WHERE rn = 1
),
with_carrier AS (
    SELECT
        t.*,
        c.carrier_name,
        c.service_level,
        c.avg_transit_days,
        c.cost_per_lb AS carrier_rate_per_lb
    FROM typed t
    LEFT JOIN bronze.raw_carriers c
        ON t.carrier_id = c.carrier_id
)
SELECT

```

```

*,
current_timestamp() AS _loaded_at
FROM with_carrier;

SELECT 'silver.silver_shipments' AS tbl, COUNT(*) AS rows FROM silver.silver_shipments;

```

Source: bronze.raw_purchase_orders + bronze.raw_suppliers (lookup)

Key derived cols: actual_lead_days, days_late

CELL 8 – silver_purchase_orders

```

CREATE OR REPLACE TABLE silver.silver_purchase_orders
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY po_id
               ORDER BY _loaded_at DESC
           ) AS rn
    FROM bronze.raw_purchase_orders
    WHERE po_id IS NOT NULL
),
typed AS (
    SELECT
        po_id,
        supplier_id,
        sku,
        warehouse_id,
        TO_DATE(po_date) AS po_date,
        TO_DATE(expected_delivery) AS expected_delivery,
        TO_DATE(actual_delivery) AS actual_delivery,
        UPPER(TRIM(status)) AS status,
        CAST(qty_ordered AS INT) AS qty_ordered,
        CAST(unit_cost AS DOUBLE) AS unit_cost,
        CAST(total_cost AS DOUBLE) AS total_cost,
        CAST(is_on_time AS BOOLEAN) AS is_on_time,
        -- how long from PO to actual receipt
        CASE
            WHEN actual_delivery IS NOT NULL
            THEN DATEDIFF(TO_DATE(actual_delivery), TO_DATE(po_date))
            ELSE NULL
        END AS actual_lead_days,
        -- positive = late vs expected, negative = early
        CASE
            WHEN actual_delivery IS NOT NULL
            AND expected_delivery IS NOT NULL
            THEN DATEDIFF(
                TO_DATE(actual_delivery),

```

```

        TO_DATE(expected_delivery))
    ELSE 0
END
        AS days_late
FROM deduped
WHERE rn = 1
),
with_supplier AS (
    SELECT
        t.*,
        s.company_name      AS supplier_name,
        s.reliability_score,
        s.is_preferred,
        s.lead_time_days    AS supplier_lead_time_days
    FROM typed t
    LEFT JOIN bronze.raw_suppliers s
        ON t.supplier_id = s.supplier_id
)
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM with_supplier;

SELECT 'silver.silver_purchase_orders' AS tbl, COUNT(*) AS rows
FROM silver.silver_purchase_orders;

```



Source: bronze.raw_returns

Key derived cols: days_to_restock, is_resellable, reason_group



CELL 9 – silver_returns

```

CREATE OR REPLACE TABLE silver.silver_returns
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY return_id
            ORDER BY _loaded_at DESC
        ) AS rn
    FROM bronze.raw_returns
    WHERE return_id IS NOT NULL
)
SELECT
    return_id,
    order_id,
    sku,
    TO_DATE(return_date)      AS return_date,
    TO_DATE(restock_date)     AS restock_date,
    CAST(qty_returned AS INT) AS qty_returned,

```

```

CAST(refund_amount AS DOUBLE)      AS refund_amount,
reason_code,
UPPER(TRIM(condition))             AS condition,
disposition,
-- how many days until item was restocked after return
CASE
    WHEN restock_date IS NOT NULL
    THEN DATEDIFF(TO_DATE(restock_date), TO_DATE(return_date))
    ELSE NULL
END                                AS days_to_restock,
-- items in new/like-new condition can go back to shelf
UPPER(TRIM(condition)) IN ('NEW', 'LIKE_NEW') AS is_resellable,
-- group reason codes into business-readable buckets
CASE
    WHEN UPPER(reason_code) IN ('DAMAGED', 'DEFECTIVE')
    THEN 'Quality'
    WHEN UPPER(reason_code) IN ('NOT_AS_DESCRIBED', 'WRONG_ITEM')
    THEN 'Accuracy'
    WHEN UPPER(reason_code) = 'CHANGED_MIND'
    THEN 'Customer'
    ELSE 'Other'
END                                AS reason_group,
current_timestamp()                AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'silver.silver_returns' AS tbl, COUNT(*) AS rows FROM silver.silver_returns;

```



Source: bronze.raw_supplier_performance + bronze.raw_suppliers (lookup)

Key derived cols: composite_score (weighted), score_tier



CELL 10 – silver_supplier_performance

```

CREATE OR REPLACE TABLE silver.silver_supplier_performance
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY perf_id
            ORDER BY _loaded_at DESC
        ) AS rn
    FROM bronze.raw_supplier_performance
    WHERE perf_id IS NOT NULL
),
scored AS (
    SELECT
        perf_id,
        supplier_id,

```

```

        CAST(on_time_rate      AS DOUBLE) AS on_time_rate,
        CAST(fill_rate        AS DOUBLE) AS fill_rate,
        CAST(defect_rate      AS DOUBLE) AS defect_rate,
        CAST(avg_lead_days    AS DOUBLE) AS avg_lead_days,
        CAST(score            AS DOUBLE) AS score,
        CAST(total_pos        AS INT)    AS total_pos,
        CAST(on_time_deliveries AS INT)    AS on_time_deliveries,
        CAST(period_month     AS INT)    AS period_month,
        CAST(period_year      AS INT)    AS period_year,
        month_label,
        -- composite = 40% on-time + 40% fill + 20% quality (1 - defect)
        (CAST(on_time_rate AS DOUBLE) * 0.4)
        + (CAST(fill_rate AS DOUBLE) * 0.4)
        + ((1.0 - CAST(defect_rate AS DOUBLE)) * 0.2) AS composite_score
    FROM deduped
    WHERE rn = 1
),
tiered AS (
    SELECT
        *,
        CASE
            WHEN composite_score >= 0.95 THEN 'A'
            WHEN composite_score >= 0.85 THEN 'B'
            ELSE 'C'
        END AS score_tier
    FROM scored
),
with_supplier AS (
    SELECT
        t.*,
        s.company_name AS supplier_name,
        s.is_preferred,
        s.country
    FROM tiered t
    LEFT JOIN bronze.raw_suppliers s
        ON t.supplier_id = s.supplier_id
)
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM with_supplier;

SELECT 'silver.silver_supplier_performance' AS tbl, COUNT(*) AS rows
FROM silver.silver_supplier_performance;

```



Source: bronze.raw_employees + bronze.raw_warehouses (lookup)

Key derived cols: tenure_days, annual_cost_est



CELL 11 – silver_employees

```

CREATE OR REPLACE TABLE silver.silver_employees
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY employee_id
               ORDER BY _loaded_at DESC
           ) AS rn
    FROM bronze.raw_employees
    WHERE employee_id IS NOT NULL
),
typed AS (
    SELECT
        employee_id,
        first_name,
        last_name,
        role,
        warehouse_id,
        TO_DATE(hire_date) AS hire_date,
        CAST(hourly_rate AS DOUBLE) AS hourly_rate,
        CAST(is_active AS BOOLEAN) AS is_active,
        -- days employed as of today
        DATEDIFF(CURRENT_DATE(), TO_DATE(hire_date)) AS tenure_days,
        -- estimated annual cost assuming 2,080 working hours/year (52wk × 40hr)
        CAST(hourly_rate AS DOUBLE) * 2080.0 AS annual_cost_est
    FROM deduped
    WHERE rn = 1
),
with_warehouse AS (
    SELECT
        t.*,
        w.name AS warehouse_name,
        w.region AS warehouse_region
    FROM typed t
    LEFT JOIN bronze.raw_warehouses w
        ON t.warehouse_id = w.warehouse_id
)
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM with_warehouse;

SELECT 'silver.silver_employees' AS tbl, COUNT(*) AS rows FROM silver.silver_employees;

```



Source: bronze.raw_labor_shifts + bronze.raw_employees (lookup)

Key derived cols: picks_per_hour, units_per_hour, labor_cost



CELL 12 – silver_labor_shifts

```

CREATE OR REPLACE TABLE silver.silver_labor_shifts
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY shift_id
            ORDER BY _loaded_at DESC
        ) AS rn
    FROM bronze.raw_labor_shifts
    WHERE shift_id IS NOT NULL
),
typed AS (
    SELECT
        shift_id,
        employee_id,
        warehouse_id,
        zone_assigned,
        shift_type,
        TO_DATE(shift_date) AS shift_date,
        CAST(hours_worked AS DOUBLE) AS hours_worked,
        CAST(picks_completed AS INT) AS picks_completed,
        CAST(units_processed AS INT) AS units_processed,
        DATE_FORMAT(TO_DATE(shift_date), 'yyyy-MM') AS shift_month,
        -- throughput rates – guard against zero division
        CASE
            WHEN CAST(hours_worked AS DOUBLE) > 0
            THEN CAST(picks_completed AS DOUBLE)
                / CAST(hours_worked AS DOUBLE)
            ELSE 0.0
        END AS picks_per_hour,
        CASE
            WHEN CAST(hours_worked AS DOUBLE) > 0
            THEN CAST(units_processed AS DOUBLE)
                / CAST(hours_worked AS DOUBLE)
            ELSE 0.0
        END AS units_per_hour
    FROM deduped
    WHERE rn = 1
),
with_employee AS (
    SELECT
        t.*,
        e.first_name,
        e.last_name,
        e.role,
        e.hourly_rate,
        -- total shift cost = hours × hourly rate from employee master
        t.hours_worked * e.hourly_rate AS labor_cost
    FROM typed t
    LEFT JOIN bronze.raw_employees e
        ON t.employee_id = e.employee_id
)
SELECT
    *,
    current_timestamp() AS _loaded_at
FROM with_employee;

SELECT 'silver.silver_labor_shifts' AS tbl, COUNT(*) AS rows FROM silver.silver_labor_shifts;

```


Source: bronze.raw_dock_activity

Key derived cols: activity_date, activity_hour, pallets_per_hour, is_inbound

CELL 13 – silver_dock_activity

```
CREATE OR REPLACE TABLE silver.silver_dock_activity
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY dock_id
               ORDER BY _loaded_at DESC
           ) AS rn
    FROM bronze.raw_dock_activity
    WHERE dock_id IS NOT NULL
)
SELECT
    dock_id,
    warehouse_id,
    carrier_id,
    shipment_id,
    UPPER(TRIM(activity_type)) AS activity_type,
    TO_TIMESTAMP(arrived_at) AS arrived_at,
    TO_TIMESTAMP(departed_at) AS departed_at,
    CAST(duration_minutes AS INT) AS duration_minutes,
    CAST(num_pallets AS INT) AS num_pallets,
    TO_DATE(arrived_at) AS activity_date,
    HOUR(TO_TIMESTAMP(arrived_at)) AS activity_hour,
    -- pallets moved per hour – guard against zero duration
    CASE
        WHEN CAST(duration_minutes AS INT) > 0
        THEN CAST(num_pallets AS DOUBLE)
            / (CAST(duration_minutes AS DOUBLE) / 60.0)
        ELSE NULL
    END AS pallets_per_hour,
    -- flag inbound receipts for dock utilization split reporting
    UPPER(TRIM(activity_type)) = 'INBOUND_RECEIVE' AS is_inbound,
    current_timestamp() AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'silver.silver_dock_activity' AS tbl, COUNT(*) AS rows FROM silver.silver_dock_activity;
```

Source: bronze.raw_locations

Key derived cols: none — clean typing and dedup only

(Not in original spec — included here for completeness)

Government	Percentage
Current government	85%
Previous government	15%

CELL 14 – silver_locations

```
CREATE OR REPLACE TABLE silver.silver_locations
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY location_id
               ORDER BY _loaded_at DESC
           ) AS rn
    FROM bronze.raw_locations
    WHERE location_id IS NOT NULL
)
SELECT
    location_id,
    warehouse_id,
    zone,
    aisle,
    bay,
    level,
    position,
    UPPER(TRIM(location_type))           AS location_type,
    CAST(max_weight_kg AS DOUBLE)         AS max_weight_kg,
    CAST(max_volume_cuft AS DOUBLE)       AS max_volume_cuft,
    CAST(is_active AS BOOLEAN)            AS is_active,
    current_timestamp()                   AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'silver.silver_locations' AS tbl, COUNT(*) AS rows FROM silver.silver_locations;
```

Group	Percentage
Men	90%
Women	86%
18-29	86%
30 and older	82%

Source: bronze.raw_date_dim

Key derived cols: month_label for consistent yyyy-MM labeling in reports

Government	Percentage
Current government	85%
Previous government	15%

CELL 15 – silver_date_dim

```

CREATE OR REPLACE TABLE silver.silver_date_dim
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY date_key
            ORDER BY _loaded_at DESC
        ) AS rn
    FROM bronze.raw_date_dim
    WHERE date_key IS NOT NULL
)
SELECT
    CAST(date_key AS INT) AS date_key,
    TO_DATE(full_date) AS full_date,
    CAST(day_of_week AS INT) AS day_of_week,
    day_name,
    CAST(week_num AS INT) AS week_num,
    CAST(month_num AS INT) AS month_num,
    month_name,
    CAST(quarter AS INT) AS quarter,
    CAST(year AS INT) AS year,
    fiscal_period,
    CAST(is_weekend AS BOOLEAN) AS is_weekend,
    CAST(is_holiday AS BOOLEAN) AS is_holiday,
    -- consistent yyyy-MM label for all fact table joins in reports
    CONCAT(
        CAST(CAST(year AS INT) AS STRING),
        '-',
        LPAD(CAST(CAST(month_num AS INT) AS STRING), 2, '0')
    ) AS month_label,
    current_timestamp() AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'silver.silver_date_dim' AS tbl, COUNT(*) AS rows FROM silver.silver_date_dim;

```

Row count + PK null + duplicate check across all 15 Silver tables.

All status column values must read '✓ OK' before running Part 3.

CELL 16 – Full Silver Validation

```

WITH audit AS (

    -- silver_orders
    SELECT 'silver_orders' AS tbl, 'order_id' AS pk,
        COUNT(*) AS rows,
        SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,
        COUNT(*) - COUNT(DISTINCT order_id) AS dupes

```

```

FROM silver.silver_orders

UNION ALL SELECT 'silver_order_lines', 'line_id',
    COUNT(*),
    SUM(CASE WHEN line_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT line_id)
FROM silver.silver_order_lines

UNION ALL SELECT 'silver_customers', 'customer_id',
    COUNT(*),
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT customer_id)
FROM silver.silver_customers

UNION ALL SELECT 'silver_products', 'sku',
    COUNT(*),
    SUM(CASE WHEN sku IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT sku)
FROM silver.silver_products

UNION ALL SELECT 'silver_inventory', 'inventory_id',
    COUNT(*),
    SUM(CASE WHEN inventory_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT inventory_id)
FROM silver.silver_inventory

UNION ALL SELECT 'silver_inventory_transactions', 'txn_id',
    COUNT(*),
    SUM(CASE WHEN txn_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT txn_id)
FROM silver.silver_inventory_transactions

UNION ALL SELECT 'silver_shipments', 'shipment_id',
    COUNT(*),
    SUM(CASE WHEN shipment_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT shipment_id)
FROM silver.silver_shipments

UNION ALL SELECT 'silver_purchase_orders', 'po_id',
    COUNT(*),
    SUM(CASE WHEN po_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT po_id)
FROM silver.silver_purchase_orders

UNION ALL SELECT 'silver_returns', 'return_id',
    COUNT(*),
    SUM(CASE WHEN return_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT return_id)
FROM silver.silver_returns

UNION ALL SELECT 'silver_supplier_performance', 'perf_id',
    COUNT(*),
    SUM(CASE WHEN perf_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT perf_id)
FROM silver.silver_supplier_performance

UNION ALL SELECT 'silver_employees', 'employee_id',
    COUNT(*),
    SUM(CASE WHEN employee_id IS NULL THEN 1 ELSE 0 END),
    COUNT(*) - COUNT(DISTINCT employee_id)
FROM silver.silver_employees

UNION ALL SELECT 'silver_labor_shifts', 'shift_id',

```

```

COUNT(*),
SUM(CASE WHEN shift_id IS NULL THEN 1 ELSE 0 END),
COUNT(*) - COUNT(DISTINCT shift_id)
FROM silver.silver_labor_shifts

UNION ALL SELECT 'silver_dock_activity', 'dock_id',
COUNT(*),
SUM(CASE WHEN dock_id IS NULL THEN 1 ELSE 0 END),
COUNT(*) - COUNT(DISTINCT dock_id)
FROM silver.silver_dock_activity

UNION ALL SELECT 'silver_locations', 'location_id',
COUNT(*),
SUM(CASE WHEN location_id IS NULL THEN 1 ELSE 0 END),
COUNT(*) - COUNT(DISTINCT location_id)
FROM silver.silver_locations

UNION ALL SELECT 'silver_date_dim', 'date_key',
COUNT(*),
SUM(CASE WHEN date_key IS NULL THEN 1 ELSE 0 END),
COUNT(*) - COUNT(DISTINCT date_key)
FROM silver.silver_date_dim

)
SELECT
tbl,
pk AS primary_key,
rows,
pk_nulls,
dupes,
CASE
WHEN pk_nulls = 0 AND dupes = 0 THEN '✓ OK'
ELSE '✗ FIX'
END AS status
FROM audit
ORDER BY tbl;
--
-- All 15 rows must show status = '✓ OK' before running Part
--

```

Response	Percentage
Yes	90%
No	10%

Response	Percentage
Yes	75%
No	25%

NOTES

Response	Percentage
Yes	85%
No	15%

RUNNING ORDER:

Cells 1-15 can run in the order shown. If you want to parallelize, cells with no inter-Silver dependencies (3,4,9,13,14,15) are safe

to run concurrently. Cells that join Bronze reference tables (2,5,7,8,10,11,12) only depend on Bronze being complete — not on other Silver tables — so they are also parallelization-safe.

DEDUPLICATION STRATEGY:

`ROW_NUMBER() OVER (PARTITION BY pk ORDER BY _loaded_at DESC)`

keeps the most recently loaded record when duplicates exist.

`WHERE rn = 1` then filters to that latest row. This is equivalent to `dropDuplicates()` in PySpark but expressed entirely in SQL and gives you audit visibility into which records were dropped.

REFERENCE JOINS FROM BRONZE (not Silver):

Silver tables join Bronze reference data (`raw_products`, `raw_carriers`, `raw_suppliers`, `raw_warehouses`, `raw_employees`) rather than their Silver equivalents. This avoids Silver-to-Silver ordering dependencies and keeps each notebook cell independently re-runnable.

PYSPARK USAGE IN PART 2: ZERO LINES.

```
#####
#####
```

■ PART 3

Gold Batch

Star schema facts, dimensions, and KPI marts

`nb_gold_build_sql`

- 12 Delta tables: 5 facts, 4 dims, 3 KPI marts
- `gold_fact_orders`: CTE pre-aggregation of 128k lines to order grain
- `OPTIMIZE + ZORDER + VACUUM` issued as pure SQL commands
- `gold_kpi_iot_summary`: daily batch aggregate from 100k IoT events
- PySpark: 0 lines

Microsoft Fabric · Notebook: nb_gold_build_sql

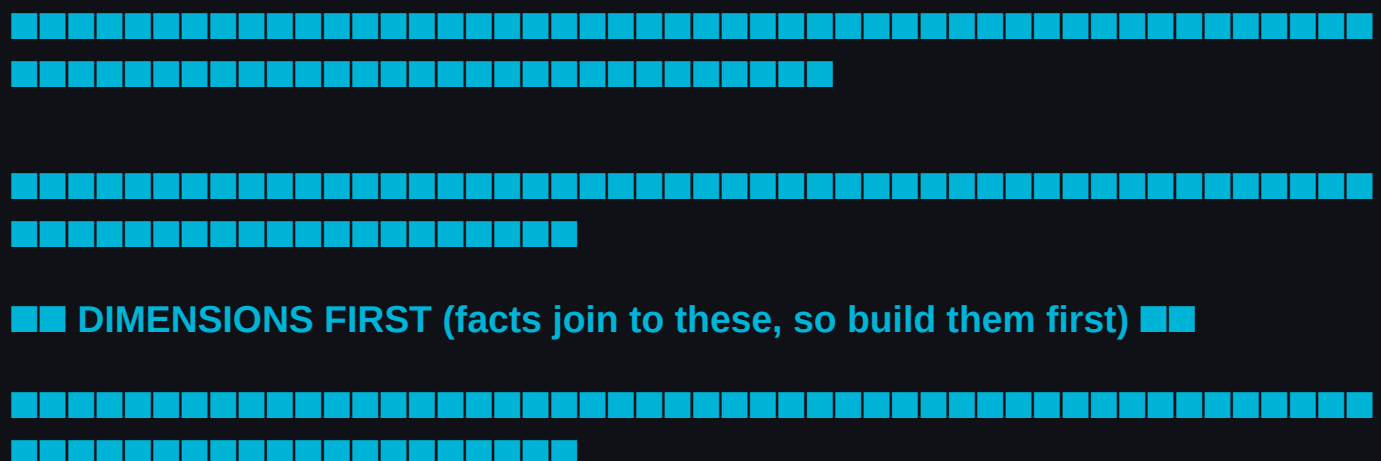
- All 12 Gold tables built via SQL CTAS from Silver (and one from Bronze).
- Aggregations use GROUP BY + window functions — no PySpark aggs.
- OPTIMIZE and ZORDER issued as SQL commands — no Python spark.sql() wrappers.
- gold_kpi_iot_summary is built here as a DAILY batch aggregate.

- Build order: Dimensions first → Facts → KPI Marts → OPTIMIZE.

**Facts : gold_fact_orders, gold_fact_inventory, gold_fact_shipments,
gold_fact_labor, gold_fact_dock**

Dims : gold_dim_customer, gold_dim_product, gold_dim_warehouse, gold_dim_date

KPI Marts: gold_kpi_supplier, gold_kpi_returns, gold_kpi_iot_summary





Source: silver.silver_date_dim

Note: This is the anchor for all time-intelligence in Power BI.

Build it first so every subsequent fact CTAS can reference it.



CELL 1 – gold_dim_date

```
CREATE OR REPLACE TABLE gold.gold_dim_date
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    date_key,
    full_date,
    day_of_week,
    day_name,
    week_num,
    month_num,
    month_name,
    month_label,
    quarter,
    year,
    fiscal_period,
    is_weekend,
    is_holiday,
    current_timestamp() AS _loaded_at
FROM silver.silver_date_dim;

SELECT 'gold.gold_dim_date' AS tbl, COUNT(*) AS rows FROM gold.gold_dim_date;
```



Source: silver.silver_customers

Derived: full_name concatenation



CELL 2 – gold_dim_customer

```
CREATE OR REPLACE TABLE gold.gold_dim_customer
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
```

```

AS
SELECT
    customer_id,
    first_name,
    last_name,
    CONCAT(first_name, ' ', last_name) AS full_name,
    segment,
    account_status,
    credit_limit,
    city,
    state,
    region,
    country,
    customer_since_year,
    created_date,
    current_timestamp()           AS _loaded_at
FROM silver.silver_customers;

SELECT 'gold.gold_dim_customer' AS tbl, COUNT(*) AS rows FROM gold.gold_dim_customer;

```



Source: silver.silver_products

Derived: price_tier bucket



CELL 3 – gold_dim_product

```

CREATE OR REPLACE TABLE gold.gold_dim_product
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    sku,
    product_name,
    category,
    subcategory,
    supplier_id,
    unit_cost,
    unit_price,
    margin_pct,
    weight_kg,
    volume_cuft,
    reorder_point,
    reorder_qty,
    lead_time_days,
    is_hazmat,
    is_perishable,
    CASE
        WHEN unit_price >= 500 THEN 'Premium'
        WHEN unit_price >= 100 THEN 'Mid'
        ELSE 'Economy'
    END           AS price_tier,

```

```

        current_timestamp()          AS _loaded_at
FROM silver.silver_products;

SELECT 'gold.gold_dim_product' AS tbl, COUNT(*) AS rows FROM gold.gold_dim_product;

```

Source: bronze.raw_warehouses (small reference table — no Silver)

CELL 4 — gold_dim_warehouse

```

CREATE OR REPLACE TABLE gold.gold_dim_warehouse
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY warehouse_id
               ORDER BY _loaded_at DESC
           ) AS rn
    FROM bronze.raw_warehouses
    WHERE warehouse_id IS NOT NULL
)
SELECT
    warehouse_id,
    name           AS warehouse_name,
    region,
    city,
    state,
    country,
    manager_id,
    CAST(capacity_sqft AS DOUBLE) AS capacity_sqft,
    CAST(num_docks     AS INT)    AS num_docks,
    CAST(num_zones     AS INT)    AS num_zones,
    current_timestamp()          AS _loaded_at
FROM deduped
WHERE rn = 1;

SELECT 'gold.gold_dim_warehouse' AS tbl, COUNT(*) AS rows FROM gold.gold_dim_warehouse;

```

FACT TABLES

Government	Percentage
Current Government	85%
Previous Government	15%

Response	Percentage
U.S. should take more action	87%
U.S. should not take more action	13%

Response	Percentage
U.S. should take more action	70%
U.S. should not take more action	30%

Source: silver_orders + silver_order_lines (aggregated) + silver_shipments (aggregated)

Grain: 1 row per order_id

Two sub-aggregations are expressed as CTEs:

line_agg — rolls 128k order line rows up to order grain

ship_agg — rolls 30k shipment rows up to order grain (first ship date, costs)

Then joined back to silver_orders for the final select.

Response	Percentage
U.S. should take more action	88%
U.S. should not take more action	12%

Response	Percentage
U.S. should take more action	75%
U.S. should not take more action	25%

CELL 5 – gold_fact_orders (primary revenue fact)

```
CREATE OR REPLACE TABLE gold.gold_fact_orders
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH line_agg AS (
  -- Aggregate 128,899 line rows to order grain
  SELECT
    order_id,
    SUM(line_value) AS total_line_value,
    SUM(line_margin) AS total_margin,
    SUM(qty_ordered) AS total_qty_ordered,
    SUM(qty_fulfilled) AS total_qty_fulfilled,
    SUM(qty_backordered) AS total_qty_backordered,
    AVG(fulfillment_rate) AS avg_fulfillment_rate,
    COUNT(DISTINCT sku) AS distinct_skus
  FROM silver.silver_order_lines
  GROUP BY order_id
),
ship_agg AS (
  -- Roll multiple shipments per order to order grain
  SELECT
    order_id,
    MIN(ship_date) AS first_ship_date,
    SUM(shipping_cost) AS total_shipping_cost,
    -- was_on_time: TRUE if ANY shipment on this order was on time
    MAX(CAST(is_on_time AS INT)) = 1 AS was_on_time,
    SUM(delay_days) AS total_delay_days,
    COUNT(shipment_id) AS num_shipments
```

```

FROM silver.silver_shipments
GROUP BY order_id
)
SELECT
    o.order_id,
    o.customer_id,
    o.warehouse_id,
    -- date_key for joining to gold_dim_date in Power BI
    CAST(DATE_FORMAT(o.order_date, 'yyyyMMdd') AS INT) AS date_key,
    o.order_date,
    o.order_month,
    o.order_year,
    o.status,
    o.priority,
    o.order_value,
    -- line-level aggregates
    l.total_line_value,
    l.total_margin,
    -- margin as fraction of line revenue
    CASE
        WHEN l.total_line_value > 0
        THEN l.total_margin / l.total_line_value
        ELSE 0.0
    END AS margin_pct,
    l.total_qty_ordered,
    l.total_qty_fulfilled,
    l.total_qty_backordered,
    l.avg_fulfillment_rate,
    l.distinct_skus,
    -- shipment aggregates
    s.first_ship_date,
    s.total_shipping_cost,
    s.was_on_time,
    s.total_delay_days,
    s.num_shipments,
    -- days from order placed to first shipment
    DATEDIFF(s.first_ship_date, o.order_date) AS days_to_ship,
    o.days_to_required,
    current_timestamp() AS _loaded_at
FROM silver.silver_orders o
LEFT JOIN line_agg l ON o.order_id = l.order_id
LEFT JOIN ship_agg s ON o.order_id = s.order_id;

SELECT 'gold.gold_fact_orders' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_orders;

```



Source: silver.silver_inventory

Grain: 1 row per inventory_id (sku × warehouse × location)

Derived: stock_status label, days_since_counted



CELL 6 – gold_fact_inventory

```
CREATE OR REPLACE TABLE gold.gold_fact_inventory
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    inventory_id,
    sku,
    warehouse_id,
    location_id,
    qty_on_hand,
    qty_reserved,
    qty_available,
    qty_in_transit,
    unit_cost,
    inventory_value,
    below_reorder,
    reorder_point,
    utilization_rate,
    last_counted_at,
    last_received_at,
    category,
    subcategory,
    product_name,
    -- how stale is the last physical count?
    DATEDIFF(CURRENT_DATE(), TO_DATE(last_counted_at)) AS days_since_counted,
    -- human-readable stock level label for Power BI slicers
    CASE
        WHEN qty_available = 0 THEN 'Out of Stock'
        WHEN below_reorder = TRUE THEN 'Low Stock'
        ELSE 'In Stock'
    END
    AS stock_status,
    current_timestamp()
    AS _loaded_at
FROM silver.silver_inventory;

SELECT 'gold.gold_fact_inventory' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_inventory;
```



Source: silver.silver_shipments

Grain: 1 row per shipment_id

Derived: date_key, ship_month



CELL 7 – gold_fact_shipments

```
CREATE OR REPLACE TABLE gold.gold_fact_shipments
```

```

USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    shipment_id,
    order_id,
    carrier_id,
    carrier_name,
    service_level,
    origin_warehouse_id,
    ship_date,
    estimated_delivery,
    actual_delivery,
    status,
    weight_lbs,
    shipping_cost,
    is_on_time,
    -- coalesce delay to 0 for clean aggregation in Power BI
    COALESCE(delay_days, 0) AS delay_days,
    num_packages,
    cost_per_lb,
    avg_transit_days,
    CAST(DATE_FORMAT(ship_date, 'yyyyMMdd') AS INT) AS date_key,
    DATE_FORMAT(ship_date, 'yyyy-MM') AS ship_month,
    current_timestamp() AS _loaded_at
FROM silver.silver_shipments;

SELECT 'gold.gold_fact_shipments' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_shipments;

```



Source: silver.silver_labor_shifts

Grain: 1 row per shift_id

Derived: date_key, cost_per_pick



CELL 8 – gold_fact_labor

```

CREATE OR REPLACE TABLE gold.gold_fact_labor
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    shift_id,
    employee_id,
    warehouse_id,
    zone_assigned,
    shift_type,
    shift_date,
    shift_month,
    hours_worked,

```

```

picks_completed,
units_processed,
picks_per_hour,
units_per_hour,
labor_cost,
role,
first_name,
last_name,
hourly_rate,
CAST(DATE_FORMAT(shift_date, 'yyyyMMdd') AS INT) AS date_key,
-- cost efficiency per individual pick
CASE
    WHEN picks_completed > 0
    THEN labor_cost / picks_completed
    ELSE NULL
END
current_timestamp()
FROM silver.silver_labor_shifts;

AS cost_per_pick,
AS _loaded_at

SELECT 'gold.gold_fact_labor' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_labor;

```



Source: silver.silver_dock_activity

Grain: 1 row per dock_id

Derived: date_key



CELL 9 – gold_fact_dock

```

CREATE OR REPLACE TABLE gold.gold_fact_dock
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    dock_id,
    warehouse_id,
    carrier_id,
    shipment_id,
    activity_type,
    activity_date,
    activity_hour,
    duration_minutes,
    num_pallets,
    pallets_per_hour,
    is_inbound,
    CAST(DATE_FORMAT(activity_date, 'yyyyMMdd') AS INT) AS date_key,
    current_timestamp()
FROM silver.silver_dock_activity;

AS _loaded_at

SELECT 'gold.gold_fact_dock' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_dock;

```


Age Group	Percentage
18-24	25%
25-34	20%
35-44	15%
45-54	10%
55-64	8%
65-74	5%
75-84	3%
85+	5%

KPI MARTS	Progress (%)
[Redacted]	100%
[Redacted]	~15%

[illegible]

Response	Percentage
Yes	75%
No	25%

Source: silver.silver_supplier_performance + bronze.raw_suppliers

Grain: 1 row per perf_id (supplier × month)

Purpose: Supplier Scorecard report page

CELL 10 – gold_kpi_supplier

```
CREATE OR REPLACE TABLE gold.gold_kpi_supplier
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    sp.perf_id,
    sp.supplier_id,
    sp.supplier_name,
    CAST(s.is_preferred AS BOOLEAN) AS is_preferred,
    sp.country,
    sp.period_month,
    sp.period_year,
    sp.month_label,
    sp.total_pos,
    sp.on_time_deliveries,
    sp.on_time_rate,
    sp.fill_rate,
    sp.defect_rate,
    sp.avg_lead_days,
    sp.composite_score,
    sp.score_tier,
    s.reliability_score,
    current_timestamp() AS _loaded_at
FROM silver.silver_supplier_performance sp
LEFT JOIN bronze.raw_suppliers s
    ON sp.supplier_id = s.supplier_id;

SELECT 'gold.gold_kpi_supplier' AS tbl, COUNT(*) AS rows FROM gold.gold_kpi_supplier;
```

Source: silver.silver_returns + silver.silver_products (lookup)

Grain: 1 row per return_id

Derived: return_month, return_year, loss_amount (non-resellable only)

CELL 11 – gold_kpi_returns

```
CREATE OR REPLACE TABLE gold.gold_kpi_returns
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
    r.return_id,
    r.order_id,
    r.sku,
    p.product_name,
    p.category,
    r.qty_returned,
    r.refund_amount,
    r.return_date,
    DATE_FORMAT(r.return_date, 'yyyy-MM')           AS return_month,
    YEAR(r.return_date)                             AS return_year,
    r.reason_code,
    r.reason_group,
    r.condition,
    r.disposition,
    r.is_resellable,
    r.days_to_restock,
    p.unit_cost,
    -- financial loss: non-resellable items written off at cost
    CASE
        WHEN r.is_resellable = FALSE
        THEN r.qty_returned * p.unit_cost
        ELSE 0.0
    END                                             AS loss_amount,
    current_timestamp()                           AS _loaded_at
FROM silver.silver_returns r
LEFT JOIN silver.silver_products p
    ON r.sku = p.sku;

SELECT 'gold.gold_kpi_returns' AS tbl, COUNT(*) AS rows FROM gold.gold_kpi_returns;
```

Source: bronze.raw_iot_events (100,000 rows)

Grain: 1 row per device_id × warehouse_id × zone × event_date

This is the DAILY batch aggregate — suitable for historical reporting.

Part 4 (streaming) produces gold_kpi_iot_stream at HOURLY window grain for near-real-time dashboard tiles.



CELL 12 – gold_kpi_iot_summary (BATCH – daily grain)

```
CREATE OR REPLACE TABLE gold.gold_kpi_iot_summary
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH cast_events AS (
    SELECT
        event_id,
        device_id,
        device_type,
        warehouse_id,
        zone,
        event_type,
        TO_DATE(event_timestamp) AS event_date,
        CAST(speed_mph AS DOUBLE) AS speed_mph,
        CAST(battery_pct AS DOUBLE) AS battery_pct,
        CAST(temperature_f AS DOUBLE) AS temperature_f,
        CAST(carrying_qty AS INT) AS carrying_qty
    FROM bronze.raw_iot_events
    WHERE event_id IS NOT NULL
)
SELECT
    device_id,
    device_type,
    warehouse_id,
    zone,
    event_date,
    CAST(DATE_FORMAT(event_date, 'yyyyMMdd') AS INT) AS date_key,
    -- volume
    COUNT(event_id) AS total_events,
    -- speed metrics
    ROUND(AVG(speed_mph), 2) AS avg_speed_mph,
    ROUND(MAX(speed_mph), 2) AS max_speed_mph,
    -- battery health
    ROUND(AVG(battery_pct), 2) AS avg_battery_pct,
    ROUND(MIN(battery_pct), 2) AS min_battery_pct,
    -- temperature
    ROUND(AVG(temperature_f), 2) AS avg_temp_f,
    -- payload
    SUM(carrying_qty) AS total_carrying_qty,
    -- event type breakdown
    SUM(CASE WHEN event_type = 'ALERT' THEN 1 ELSE 0 END) AS alert_count,
    SUM(CASE WHEN event_type = 'IDLE' THEN 1 ELSE 0 END) AS idle_count,
```

```
SUM(CASE WHEN event_type = 'MOVE' THEN 1 ELSE 0 END) AS move_count,
-- alert rate as fraction of daily events
ROUND(
    SUM(CASE WHEN event_type = 'ALERT' THEN 1 ELSE 0 END)
    / CAST(COUNT(event_id) AS DOUBLE),
    4
) AS alert_rate,
current_timestamp() AS _loaded_at
FROM cast_events
GROUP BY
    device_id,
    device_type,
    warehouse_id,
    zone,
    event_date;

SELECT 'gold.gold_kpi_iot_summary' AS tbl, COUNT(*) AS rows
FROM gold.gold_kpi_iot_summary;
```

Response	Percentage
U.S. should take more action to reduce global warming	90%
U.S. should not take more action to reduce global warming	10%

Response	Percentage
U.S. should take more action to reduce global warming	75%
U.S. should not take more action to reduce global warming	25%

■ POST-BUILD: OPTIMIZE + ZORDER + VACUUM

[illegible]

Run these cells AFTER all 12 Gold tables are written.

OPTIMIZE compacts small Parquet files; ZORDER co-locates rows by

the columns Power BI filters on most — dramatically cuts scan time.

VACUUM removes old snapshots (keeps 7-day / 168-hour history).

Response	Percentage
U.S. should take more action	90%
U.S. should not take more action	10%

Response	Percentage
U.S. should take more action	75%
U.S. should not take more action	25%

Response	Percentage
Yes	90%
No	10%

Response	Percentage
Yes	75%
No	25%

Response	Percentage
U.S. should take more action	90%
U.S. should not take more action	10%

Response	Percentage
U.S. should take more action	75%
U.S. should not take more action	25%

CELL 13 – OPTIMIZE + ZORDER: Fact Tables

```
OPTIMIZE gold.gold_fact_orders ZORDER BY (order_date, customer_id);
OPTIMIZE gold.gold_fact_inventory ZORDER BY (sku, warehouse_id);
OPTIMIZE gold.gold_fact_shipments ZORDER BY (ship_date, carrier_id);
OPTIMIZE gold.gold_fact_labor ZORDER BY (shift_date, warehouse_id);
OPTIMIZE gold.gold_fact_dock ZORDER BY (activity_date, warehouse_id);
```

CELL 14 – OPTIMIZE + ZORDER: KPI Marts

```

OPTIMIZE gold.gold_kpi_supplier      ZORDER BY (period_year,  period_month);
OPTIMIZE gold.gold_kpi_returns       ZORDER BY (return_date,   reason_code);
OPTIMIZE gold.gold_kpi_iot_summary   ZORDER BY (event_date,    warehouse_id);

```

CELL 15 – VACUUM: All Gold Tables (retain 7 days)

```

VACUUM gold.gold_fact_orders      RETAIN 168 HOURS;
VACUUM gold.gold_fact_inventory   RETAIN 168 HOURS;
VACUUM gold.gold_fact_shipments   RETAIN 168 HOURS;
VACUUM gold.gold_fact_labor       RETAIN 168 HOURS;
VACUUM gold.gold_fact_dock        RETAIN 168 HOURS;
VACUUM gold.gold_dim_customer     RETAIN 168 HOURS;
VACUUM gold.gold_dim_product      RETAIN 168 HOURS;
VACUUM gold.gold_dim_warehouse    RETAIN 168 HOURS;
VACUUM gold.gold_dim_date         RETAIN 168 HOURS;
VACUUM gold.gold_kpi_supplier     RETAIN 168 HOURS;
VACUUM gold.gold_kpi_returns      RETAIN 168 HOURS;
VACUUM gold.gold_kpi_iot_summary   RETAIN 168 HOURS;

```

CELL 16 – Full Gold Validation (row counts, all 12 tables)

```

SELECT tbl, rows,
       CASE WHEN rows > 0 THEN '✓ OK' ELSE '✗ EMPTY' END AS status
FROM (
  SELECT 'gold_fact_orders'      AS tbl, COUNT(*) AS rows FROM gold.gold_fact_orders
  UNION ALL
  SELECT 'gold_fact_inventory',   COUNT(*) FROM gold.gold_fact_inventory
  UNION ALL
  SELECT 'gold_fact_shipments',   COUNT(*) FROM gold.gold_fact_shipments
  UNION ALL
  SELECT 'gold_fact_labor',       COUNT(*) FROM gold.gold_fact_labor
  UNION ALL

```

[illegible]

Government	Percentage
Current Government	85%
Previous Government	15%

Run after validation to confirm the core KPIs look reasonable.

[illegible]

CELL 17 – Quick Sanity: Revenue + OTD Rate

```
SELECT
    ROUND(SUM(order_value),          2) AS total_revenue,
    ROUND(AVG(margin_pct) * 100,     2) AS avg_margin_pct,
    COUNT(*)                          AS total_orders,
    SUM(CASE WHEN was_on_time THEN 1 ELSE 0 END) AS orders_on_time,
    ROUND(
        SUM(CASE WHEN was_on_time THEN 1 ELSE 0 END)
        / CAST(COUNT(*) AS DOUBLE) * 100, 2
    ) AS otd_rate_pct
FROM gold.gold_fact_orders;
```

Demographic Group	Percentage
Men	90%
Women	85%
18-29	85%
30 and older	75%

NOTES

Response	Percentage
U.S. should take more action	90%
U.S. should not take more action	10%

Response	Percentage
U.S. should take more action	75%
U.S. should not take more action	25%

BUILD ORDER:

gold_fact_orders CTE DESIGN:

INCREMENTAL REFRESH (production):

PYSPARK USAGE IN PART 3: ZERO LINES.

Total PySpark across Parts 1-3: 3 lines (schema bootstrap only).

Government	Percentage
Current government	85%
Previous government	15%

■ PART 4

IoT Streaming + Pipeline

Delta readStream → SQL 1-hour window → writeStream

```
nb_iot_stream_sql
```

- Reads bronze.raw_iot_events as Delta stream (readStream)
- SQL tumbling window aggregates events into 1-hour buckets
- writeStream appends to gold.gold_kpi_iot_stream continuously
- Checkpoint-based resume — no duplicate rows on restart
- Data Factory pipeline: Bronze → Silver → Gold → PBI refresh
- PySpark: ~15 lines (readStream/writeStream wrappers only)

WMS SQL-First Medallion — Part 4 of 4: STREAMING + PIPELINE

Microsoft Fabric · Notebook: nb_iot_stream_sql

What this notebook does:

1. Reads bronze.raw_iot_events as a Delta stream (readStream)
2. Registers the stream as a temp view so pure SQL can query it
3. A SQL tumbling window aggregates events into 1-hour buckets
4. writeStream writes results continuously to gold.gold_kpi_iot_stream

Prerequisites:

- bronze.raw_iot_events must exist with delta.enableChangeDataFeed = true (set in Part 1, Cell 19)
- Parts 1-3 must have completed successfully
- This notebook runs CONTINUOUSLY (streaming) — it is NOT part of the daily batch pipeline. Wire it as a separate always-on job in Fabric.

PySpark usage: ~15 lines — unavoidable for readStream/writeStream setup.

All aggregation and transformation logic is SQL.

```
#####
#####
```

```
#####
#####
```

This creates the empty Delta table that writeStream will write into.

Schema is declared explicitly — streaming sinks require a fixed schema and cannot use inferSchema. Run this cell manually before starting

the stream for the first time. Skip on subsequent restarts.



CELL 1 – Create the streaming target table (SQL, run once)

```
CREATE TABLE IF NOT EXISTS gold.gold_kpi_iot_stream
(
    window_start      TIMESTAMP,
    window_end        TIMESTAMP,
    device_id          STRING,
    device_type        STRING,
    warehouse_id       STRING,
    zone               STRING,
    total_events        BIGINT,
    avg_speed_mph       DOUBLE,
    max_speed_mph       DOUBLE,
    avg_battery_pct     DOUBLE,
    min_battery_pct     DOUBLE,
    avg_temp_f          DOUBLE,
    total_carrying_qty BIGINT,
    alert_count         BIGINT,
    idle_count          BIGINT,
    move_count          BIGINT,
    alert_rate          DOUBLE,
    date_key            INT,
    event_hour          INT,
    _loaded_at         TIMESTAMP
)
USING DELTA
TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact'   = 'true'
);

-- Confirm it exists
DESCRIBE TABLE gold.gold_kpi_iot_stream;
```



This is the only Python in the entire 4-part project beyond the 3-line schema bootstrap in Part 1.

readStream reads only NEW rows added to bronze.raw_iot_events since the last checkpoint — it does not reprocess the full 100k on restart.



CELL 2 – Read the Delta stream + register as temp view (Python)

```
from pyspark.sql import functions as F

# ■■■ 1. Read bronze.raw_iot_events as a Delta stream ■■■■■■■■■■■■■■■■
#   maxFilesPerTrigger=1 processes one new file batch per micro-batch.
#   Increase for higher throughput; lower for lower latency.
iot_stream = (
    spark.readStream
        .format("delta")
        .option("maxFilesPerTrigger", 1)
        .table("bronze.raw_iot_events")
)

# ■■■ 2. Cast event_timestamp to TimestampType for window() ■■■■■■■■■■■■■■■■
#   window() requires a proper Timestamp column – string won't work.
iot_typed = iot_stream.withColumn(
    "event_ts", F.to_timestamp("event_timestamp")
)

# ■■■ 3. Register as a temp view so SQL cells can query it ■■■■■■■■■■■■■■■■
iot_typed.createOrReplaceTempView("iot_stream_view")

print("Stream registered as temp view: iot_stream_view")
print("Schema:")
iot_typed.printSchema()
```

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	5
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	---

window() in Spark SQL creates tumbling time windows over the stream.

'1 hour' means each output row covers exactly one 60-minute bucket.

window_start and **window_end** are the bucket boundaries.

This view is **LAZY** — it defines the query but does not execute it.

Execution happens in Cell 4 when writeStream starts consuming it.

Response	Percentage
U.S. should take more action	93%
U.S. should not take more action	7%

Response	Percentage
U.S. should take more action	83%
U.S. should not take more action	15%

CELL 3 – SQL tumbling window aggregation

```
CREATE OR REPLACE TEMPORARY VIEW iot_windowed AS
SELECT
    -- tumbling 1-hour window boundaries
    window(event_ts, '1 hour').start           AS window_start,
    window(event_ts, '1 hour').end             AS window_end,

    device_id,
    device_type,
    warehouse_id,
    zone,
```

```

-- event volume within this hour window
COUNT(event_id)                                AS total_events,

-- speed metrics
ROUND(AVG(CAST(speed_mph AS DOUBLE)), 2)         AS avg_speed_mph,
ROUND(MAX(CAST(speed_mph AS DOUBLE)), 2)         AS max_speed_mph,

-- battery health
ROUND(AVG(CAST(battery_pct AS DOUBLE)), 2)       AS avg_battery_pct,
ROUND(MIN(CAST(battery_pct AS DOUBLE)), 2)       AS min_battery_pct,

-- temperature
ROUND(AVG(CAST(temperature_f AS DOUBLE)), 2)     AS avg_temp_f,

-- payload weight carried
SUM(CAST(carrying_qty AS INT))                   AS total_carrying_qty,

-- event type breakdown for the hour
SUM(CASE WHEN event_type = 'ALERT' THEN 1 ELSE 0 END) AS alert_count,
SUM(CASE WHEN event_type = 'IDLE' THEN 1 ELSE 0 END) AS idle_count,
SUM(CASE WHEN event_type = 'MOVE' THEN 1 ELSE 0 END) AS move_count,

-- alert rate: what fraction of this hour's events were alerts
ROUND(
    SUM(CASE WHEN event_type = 'ALERT' THEN 1 ELSE 0 END)
    / CAST(COUNT(event_id) AS DOUBLE),
    4
)                                                  AS alert_rate,

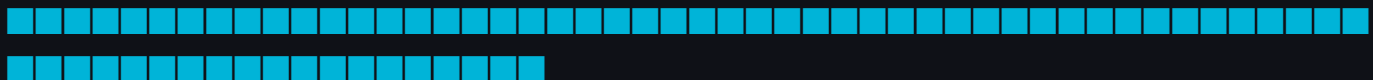
-- date_key for joining to gold_dim_date in Power BI
CAST(
    DATE_FORMAT(window(event_ts, '1 hour').start, 'yyyyMMdd')
AS INT)                                           AS date_key,

-- hour of day (0-23) for heatmap visuals
HOUR(window(event_ts, '1 hour').start)          AS event_hour,

current_timestamp()                             AS _loaded_at

FROM iot_stream_view
GROUP BY
    window(event_ts, '1 hour'),
    device_id,
    device_type,
    warehouse_id,
    zone;

```



outputMode("append"): each completed 1-hour window writes one row.

Use "append" not "complete" — complete would rewrite the entire table every micro-batch, which is expensive at scale.

trigger(processingTime="5 minutes"): micro-batch runs every 5 min.

Tune down to "1 minute" for lower latency tiles in Power BI.

checkpointLocation: Fabric OneLake path for stream state. If the stream restarts it resumes from exactly where it left off — no duplicate rows, no lost events.



CELL 4 – writeStream: write windowed results to Gold (Python)

```
CHECKPOINT_PATH = "Files/checkpoints/iot_stream"

query = (
    spark.sql("SELECT * FROM iot_windowed")
        .writeStream
        .format("delta")
        .outputMode("append")
        .trigger(processingTime="5 minutes")
        .option("checkpointLocation", CHECKPOINT_PATH)
        .toTable("gold.gold_kpi_iot_stream")
)

print(f"Stream started → gold.gold_kpi_iot_stream")
print(f"Checkpoint path → {CHECKPOINT_PATH}")
print(f"Trigger → every 5 minutes")
print(f"Query ID → {query.id}")
print(f"Query name → {query.name}")
print("\nStream is running. Use Cell 5 to monitor. Cell 6 to stop.")
```



CELL 5 – Monitor stream progress (Python – run anytime)

```
import json

if query.isActive:
    progress = query.lastProgress
    if progress:
        print(f"Status : {query.status['message']}")
        print(f"Trigger : {progress.get('trigger', {})}")
        print(f"Input rows/sec : {progress.get('inputRowsPerSecond', 0):.2f}")
        print(f"Processed rows : {progress.get('numInputRows', 0)}")
        print(f"Batch duration : {progress.get('durationMs', {}).get('triggerExecution', 0)} ms")
        print(f"Watermark : {progress.get('eventTime', {}).get('watermark', 'none')}")
    else:
```

```

        print("Stream active – no batch completed yet. Wait for first trigger.")
    else:
        print(f"Stream stopped. Last exception: {query.exception()}")

```

```

████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████

```

```

████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████

```

CELL 6 – Graceful stop (Python – run to stop the stream cleanly)

```

query.stop()
print("Stream stopped gracefully.")

```

```

████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████

```

Query the live gold table while the stream is running.

```

████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████

```

CELL 7 – Spot-check streamed results (SQL – run anytime)

```

-- Latest 20 hourly windows written to gold
SELECT
    window_start,
    window_end,
    device_id,
    device_type,
    warehouse_id,
    zone,
    total_events,
    avg_speed_mph,
    avg_battery_pct,
    alert_count,
    alert_rate,
    event_hour
FROM gold.gold_kpi_iot_stream
ORDER BY window_start DESC
LIMIT 20;

```

```

████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████

```

```

████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████

```

CELL 8 – Alert summary by warehouse (SQL – Power BI-ready query)

```
-- Hourly alert rate by warehouse for the most recent 24 hours
SELECT
    warehouse_id,
    zone,
    window_start,
    event_hour,
    SUM(alert_count)                AS total_alerts,
    SUM(total_events)              AS total_events,
    ROUND(SUM(alert_count) / SUM(total_events) * 100, 2) AS alert_rate_pct,
    ROUND(AVG(avg_battery_pct), 2) AS avg_battery,
    ROUND(MIN(min_battery_pct), 2) AS lowest_battery
FROM gold.gold_kpi_iot_stream
WHERE window_start >= CURRENT_TIMESTAMP() - INTERVAL 24 HOURS
GROUP BY warehouse_id, zone, window_start, event_hour
ORDER BY window_start DESC, alert_rate_pct DESC;
```

Confirms the stream is picking up what the batch gold table expects.

CELL 9 – Batch vs Stream row count comparison (SQL – sanity check)

```
SELECT
    'Batch (daily grain)' AS source,
    COUNT(*)              AS rows,
    MIN(event_date)       AS earliest,
    MAX(event_date)       AS latest
FROM gold.gold_kpi_iot_summary

UNION ALL

SELECT
    'Stream (hourly grain)' AS source,
    COUNT(*)               AS rows,
    MIN(DATE(window_start)) AS earliest,
    MAX(DATE(window_start)) AS latest
FROM gold.gold_kpi_iot_stream;
```

Run manually or on a schedule — do NOT run while stream is active.

Stop the stream first (Cell 6), optimize, then restart (Cell 4).

CELL 10 – Optional: OPTIMIZE the stream table (SQL)

```
OPTIMIZE gold.gold_kpi_iot_stream ZORDER BY (window_start, warehouse_id);  
VACUUM gold.gold_kpi_iot_stream RETAIN 168 HOURS;
```



■ DATA FACTORY PIPELINE — Full Orchestration

Pipeline name: pl_wms_medallion

Wire all 4 notebooks in Fabric Data Factory as shown below.

The batch chain (Bronze → Silver → Gold) runs daily at 06:00 UTC.

The streaming notebook runs as a separate always-on job.



DAILY BATCH PIPELINE (pl_wms_medallion):



■ Schedule Trigger ■ Daily 06:00 UTC



■ Bronze Ingest ■ nb_bronze_ingest_sql Timeout: 45 min

■ (nb_bronze_...) ■



on success



■ Silver Transform ■ nb_silver_transform_sql Timeout: 60 min

■ (nb_silver_...) ■



on success



■ Gold Build ■ nb_gold_build_sql Timeout: 60 min

■ (nb_gold_...) ■



on success



■ Wait (5 min) ■ Lets Delta log settle before PBI refresh



on success



[illegible]

■ Power BI Refresh ■ Semantic model refresh activity

■ (dim tables only) ■ gold_dim_* tables in Import mode

[illegible]

STREAMING JOB (separate — always on):

[illegible]

■ nb_iot_stream_sql ■

■ Run as: Fabric Notebook Job ■

■ **Retry policy: restart on fail** ■

■ Max retries: unlimited ■

■ Checkpoint handles resume ■

[illegible][illegible][illegible]

PIPELINE SETUP STEPS (Fabric Data Factory UI):

1. Workspace → New → Data pipeline → name it pl_wms_medallion

2. Drag four Notebook activities onto the canvas:

Activity 1: "Bronze Ingest" → nb bronze ingest sql, timeout 45m

Activity 2: "Silver Transform" → nb_silver_transform_sql, timeout 60m

Activity 3: "Gold Build" → nb_gold_build_sql, timeout 60m

Activity 4: "Wait 5m" → Wait activity, 300 seconds

3. Connect on-success arrows:

Bronze Ingest → Silver Transform → Gold Build → Wait 5m

4. (Optional) Add a Power BI Dataset Refresh activity after Wait 5m

→ **select your Fabric workspace and semantic model**

→ check "Refresh dim tables" only (fact tables use DirectLake)

5. Failure alerts:

Pipeline canvas → Properties → Alerts → On Failure

→ add email recipients for ops team

6. Schedule trigger:

Pipeline canvas → Add trigger → New/Edit → Schedule

→ **Recurrence: Daily at 06:00 UTC**

→ **Save and Publish**

7. Streaming notebook (separate):

Workspace → nb_iot_stream_sql → Run → Run as job

→ **Enable "Restart on failure" in job settings**

This keeps the stream alive independently of the daily batch.

Response	Percentage
U.S. should take more action	90%
U.S. should not take more action	10%

Response	Percentage
U.S. should take more action	75%
U.S. should not take more action	25%

Response	Percentage
U.S. should take more action to reduce greenhouse gas emissions	93%
U.S. should not take more action to reduce greenhouse gas emissions	7%

Response	Percentage
U.S. should take more action to reduce greenhouse gas emissions	82%
U.S. should not take more action to reduce greenhouse gas emissions	18%

■ POWER BI — STREAMING TILE SETUP

The stream table (gold_kpi_iot_stream) is best surfaced in Power BI via DirectLake — it queries OneLake directly with no refresh needed.

Recommended visuals for the IoT / Equipment report page:

Visual Measure / Field



Line chart avg_battery_pct by window_start (last 24h)

Bar chart alert_count by zone

Card Latest alert_rate (LASTNONBLANK on window_start)

Heatmap matrix event_hour (rows) × warehouse_id (cols) → total_events

Scatter avg_speed_mph vs avg_battery_pct by device_type

DAX for streaming tile — Latest Hour Alert Rate:

Latest Alert Rate =

```
CALCULATE(
    AVERAGE(gold_kpi_iot_stream[alert_rate]),
    LASTNONBLANK(
        gold_kpi_iot_stream>window_start],
        COUNTROWS(gold_kpi_iot_stream)
    )
)
```

DAX for battery warning flag:

Low Battery Devices =

```
CALCULATE(
    DISTINCTCOUNT(gold_kpi_iot_stream>device_id]),
```

```
gold_kpi_iot_stream[min_battery_pct] < 0.2
```

)

Response	Percentage
Yes	90%
No	10%

Response	Percentage
Yes	75%
No	25%

Two horizontal bar charts showing the percentage of respondents who believe the U.S. should take more action to address climate change. The top chart shows 90% for Democrats and 75% for Republicans. The bottom chart shows 85% for Democrats and 65% for Republicans.

■ FULL PROJECT SUMMARY

Government	Percentage
Current government	85%
Previous government	15%

Part Notebook Cells SQL Python Tables Written

Age Group	Percentage
18-24	~12%
25-34	~28%
35-44	~22%
45-54	~18%
55-64	~15%
65-74	~10%
75-84	~5%
85+	~2%

1 nb_bronze_ingest_sql 21 20 1 (3 lines) 19 Bronze

2 nb_silver_transform_sql 16 16 0 15 Silver

3 nb_gold_build_sql 17 17 0 12 Gold (batch)

4 nb_iot_stream_sql 10 6 4 (~15 lines) 1 Gold (stream)

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 104

Total 64 59 5 (~18 lines) 47 tables

PySpark lines across entire project: ~18

- 3 lines: schema bootstrap (Part 1)
- 15 lines: readStream setup + writeStream + monitor/stop (Part 4)

Everything else — all typing, dedup, joining, aggregation,

optimization, validation — is pure SQL.

